

Final Architecture Mapping For Scale

Purpose

This document is the final architecture mapping reference for the current Googer system.

It is written so that:

- a developer can understand the current architecture quickly
- a client or stakeholder can understand why the structure is scalable
- future engineers can see the path from current architecture to high-scale production
- the team can avoid vague "vibe-coded" architecture claims

This document is based on the current repository structure and completed backend modularization work.

Executive Summary

Current Architecture Name

Microservice-Ready Modular Monolith

Short Meaning

- one main backend runtime still exists
- backend domains are now separated into explicit modules
- frontend, backend, worker, realtime, admin, and shared utilities already exist as distinct runtime concerns
- the codebase is prepared for later service extraction without requiring a full rewrite

Final High-Level Answer

Yes, this architecture **can be built toward**:

- **10,000+ DAU**
- **5,000+ concurrent users**

But that result depends on:

- production infrastructure

- managed database design
- object storage/CDN for media
- Redis and queue activation
- indexing and query tuning
- load testing
- observability

So the correct statement is:

> The current architecture is structurally good and can support the target growth path. > It is not the final scaling setup by itself, but it is a correct foundation for building that setup.

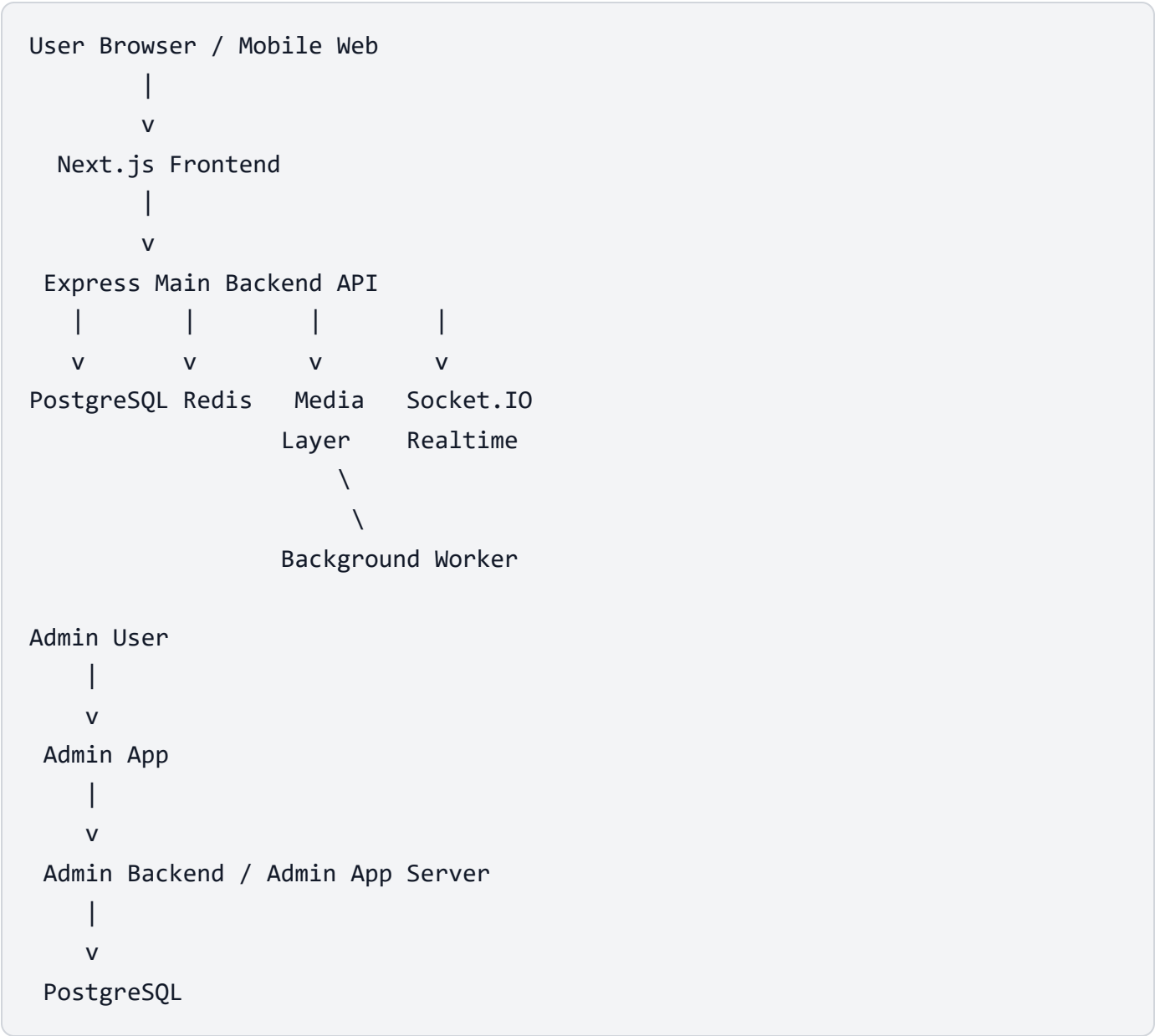
Current Architecture Reality

The current system is **not backend-less** and **not a fake frontend-only system**.

It currently contains:

- user-facing Next.js frontend
- Express backend API
- admin application
- PostgreSQL-backed business logic
- Socket.IO realtime layer
- worker/background job support
- Docker deployment preparation
- modular backend boundaries for major domains

Current Architecture Diagram



Current Backend Domain Map

The main backend now has explicit module boundaries for these domains:

```
ads
auth
chat
feed
marketplace
media
notifications
subscriptions
users
verification
```

This is important because scalability problems usually appear when domains are mixed into one large controller layer. That is no longer the main structure here.

Current Backend Responsibility Map

```
Express Routes
  |
  v
Domain Controllers
  |
  v
Domain Services
  |
  v
Domain Repositories
  |
  v
PostgreSQL
```

Domain Services also talk to:

- Redis
- Media provider
- Queue / Worker path
- External providers

Current Deployment Shape

The current codebase supports this runtime shape:

```
Web Client -----> Main Frontend Container -----> Main Backend
Container
Admin Browser -----> Admin App Container -----> Main Backend
Container

Main Backend Container -----> PostgreSQL
Main Backend Container -----> Redis
Main Backend Container -----> Object Storage / CDN

Worker Container -----> PostgreSQL
Worker Container -----> Redis
Worker Container -----> Object Storage / CDN
```

Why The Current Architecture Is Good

1. Domain Boundaries Exist

The code is no longer only route-to-large-controller spaghetti. Major domains are separated into modules, which is the correct preparation for scaling.

2. The System Already Has a Real Backend

The backend handles:

- auth
- marketplace
- ads
- wallet-related flows
- orders
- chat
- feed
- verification
- subscriptions

That means this is a real application architecture, not a front-only shell.

3. Realtime and Worker Concerns Already Exist

Chat and background job support are already part of the system shape. That matters because high-concurrency products usually need both.

4. Deployment Separation Already Exists

The code is already prepared to separate:

- frontend traffic
- API traffic
- worker traffic
- admin traffic

That is a strong base for future scaling.

5. Microservice Path Is Clear

Because modules now exist, later extraction can happen by domain instead of rewriting everything.

What This Architecture Is Not Yet

To stay accurate:

- it is **not** already full microservices
- it is **not** automatically scaled to 5,000 concurrent users today
- it is **not** complete production infrastructure by itself
- it is **not** proof of scale until load-tested

Those are not hidden mistakes. They are normal realities of a system at this stage.

Can This Current Architecture Reach 10,000 DAU and 5,000+ Concurrent Users?

Short Answer

Yes, it can be built to reach that target.

Honest Engineering Answer

The current architecture does **not** contain an obvious structural blocker that would make that goal impossible.

That means:

- the current architecture is **buildable**
- the current architecture is **scalable with the right production setup**
- the current architecture is **not wrongly designed for future growth**

What Actually Decides Success

Reaching that scale depends less on the words "microservices" and more on these production factors:

1. database design 2. read/write hot path optimization 3. queue-based background work 4. object storage and CDN for media 5. Redis for cache and distributed state 6. replica strategy for backend and frontend 7. monitoring and alerting 8. load testing with real traffic patterns

Scale-Ready Target Architecture

This is the recommended target production architecture for the current codebase:

Users / Admins

|

v

CDN / Load Balancer / Gateway

|

+++> Frontend Replica 1

+++> Frontend Replica 2

+++> Backend Replica 1

+++> Backend Replica 2

+++> Backend Replica 3

+++> Backend Replica 4

+++> Admin App

Backend Replicas connect to:

- Managed PostgreSQL
- Managed Redis
- Object Storage / CDN
- Monitoring / Logs / Metrics

Worker Replicas connect to:

- Managed PostgreSQL
- Managed Redis
- Object Storage / CDN
- Monitoring / Logs / Metrics

Recommended Future Service Extraction Order

If the team later wants real microservices, this is the safest order:

Phase 1

- keep current modular monolith
- do not split finance-sensitive areas first

Phase 2

extract first:

- media service
- notification service

- chat service

Phase 3

extract next:

- feed service
- ads service if traffic justifies it

Phase 4

extract last:

- wallet/order/payment-sensitive service

This order reduces business risk.

Production Request Flow Diagram

Normal read flow:

User

- > CDN / Load Balancer
- > Frontend
- > Backend API
- > Redis check if needed
- > PostgreSQL
- > Backend API response
- > Frontend render

Heavy async flow:

User

- > Frontend
- > Backend API
- > Queue / Redis
- > Worker
- > Object Storage / CDN
- > PostgreSQL metadata update

What Must Still Be Done For Real Scale

Even though the architecture shape is good, these items still matter before claiming high-scale production readiness:

1. Media Must Be Fully Externalized

Use:

- Cloudinary
- AWS S3
- Cloudflare R2

with CDN delivery.

2. Redis Must Be Active, Not Just Present

Use Redis for:

- queues
- cache
- Socket.IO adapter
- distributed rate limiting

3. Worker Flow Must Be Fully Used

Move heavy work into background jobs:

- media compression
- notification fanout
- report generation
- subscription renewals

4. PostgreSQL Must Be Managed and Tuned

Use:

- managed PostgreSQL
- indexes for hot queries
- tuned connection pools
- backups and failover planning

5. Observability Must Exist

Need:

- structured logs
- metrics
- uptime checks
- queue monitoring
- slow query visibility

6. Load Testing Must Prove the Claim

Do not claim 5,000+ concurrent users without tests.

Test:

- feed read load
- marketplace read load
- ad delivery load
- chat socket concurrency
- upload spikes
- wallet/order hot paths

Hidden Mistake Check

Based on the current repository structure, there is **no obvious architecture-level mistake** that makes the design fundamentally wrong.

That said, no responsible engineer should claim:

- zero risk
- zero future refactoring
- guaranteed scale without testing

The correct statement is:

> There is no obvious hidden architecture disaster in the current structure. > The current structure is valid, understandable, and scalable with the right production implementation.

Final Mapping Statement

If someone reads this architecture correctly, the message should be:

> This system is already structured as a microservice-ready modular monolith. > It has real backend architecture, real domain separation, deployment separation, realtime support, worker support, and a clear path to production scaling. > With managed infrastructure, Redis, object storage, observability, and load testing, it can be built toward 10,000+ DAU and 5,000+ concurrent users.

One-Line Final Verdict

Yes, this current architecture can be built into a high-scale production system, and the current code structure is a good foundation rather than a bad one.